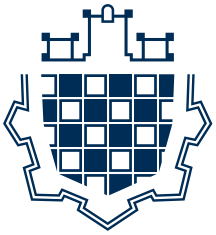

UNIVERSITY OF MANNHEIM

School of Business Informatics and Mathematics

Test Suite Minimisation in LASSO Using Stimulus-Response Matrices

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk
laith.sandouk@students.uni-mannheim.de
born May 15th, 2005
in Mannheim

Student ID Number: 1993811

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: http://swt.informatik.uni-mannheim.de

Abstract

Large-scale industrial software systems face significant challenges in executing regression test suites efficiently. In a commercial SAP environment, Bach et al. report test suites comprising more than 130,000 tests requiring hundreds of hours of sequential runtime [2]. Such execution times render continuous testing computationally infeasible and economically prohibitive. The study further revealed extensive redundancy within these test suites, implying that substantial execution cost yields only marginal additional coverage benefit. However, this is not an isolated case—test-suite redundancy is a widespread and well-documented issue in industrial software development. While this computational burden cannot be eliminated entirely, its impact can be substantially reduced through test-suite minimisation, which systematically removes redundant tests with respect to defined adequacy criteria such as statement coverage, branch coverage, or mutation score [21]. However, naive reduction approaches risk compromising fault-detection effectiveness, as tests with unique defect-revealing capabilities may be inadvertently excluded.

This thesis investigates test-suite minimisation within the *Large-Scale Software Observatorium* (LASSO) [13], a research platform for automated selection, execution, and comparative analysis of software systems. LASSO integrates open-source repositories and combines static and dynamic analyses within a unified framework designed for language independence; its current implementation focuses on Java and Python systems and enables empirically grounded studies of *Big Code* ecosystems—large, diverse corpora of source code enriched with metadata such as bug-fix histories, version control traces, and execution outcomes [1].

Our approach leverages *Stimulus–Response Matrices* (SRMs) [13], a two-dimensional representation mapping test stimuli to observed execution outcomes. We augment SRM entries with per-test coverage and mutation information, enabling minimisation algorithms to operate on fine-grained behavioural data rather than language-specific code structures.

Through a targeted literature survey encompassing greedy heuristics, exact optimisation, search-based, and data-driven minimisation strategies, we comparatively assessed candidate algorithms along seven established criteria: SRM compatibility, coverage retention, reduction effectiveness, fault-detection retention, mutation-coverage preservation, computational scalability, and implementation feasibility. The Mutation-Aware Enhanced HGS (MA-EHGS) algorithm emerged as the most promising candidate. MA-EHGS extends the empirically validated Enhanced HGS baseline [6] by integrating mutation weighting via a configurable β-parameter to balance coverage preservation and fault-detection power [10].

We successfully integrated MA-EHGS into LASSO and validated the implementation on real-world SRMs. Our evaluation achieved reductions within the ranges reported in prior studies, while maintaining high coverage and mutation-scores [6, 10]. The algorithm executes in $O(nm)$ time, ensuring computational feasibility for large-scale software analysis workflows.

Contents

List of Figures

List of Tables

List of Abbreviations

CFG	Control Flow Graph
CI	Continuous Integration
EHGS	Enhanced HGS
GA	Genetic Algorithm
HGS	Harrold–Gupta–Soffa algorithm
ILP	Integer Linear Programming
LASSO	Large-Scale Software Observatorium
LSL	LASSO Scripting Language
MA-EHGS	Mutation-Aware Enhanced HGS
QIGA	Quantum-Inspired Genetic Algorithm
SRM	Stimulus-Response Matrix
TSM	Test Suite Minimisation

1. Introduction

Software testing ensures program modifications do not introduce defects. As systems evolve, test suites expand, increasing execution cost and maintenance effort. Suites can combine human-written tests, automatically generated tests, and, increasingly in recent years, AI-generated tests. Tools like Randoop [18] and EvoSuite [5] generate thousands of cases. In large projects, execution may require days or weeks [19]. While comprehensive suites achieve high coverage and mutation scores, they frequently contain redundant tests whose requirements are subsumed by others, causing prolonged execution, increased costs, and elevated maintenance overhead.

Test-suite minimisation systematically removes redundant tests while preserving fault-detection capability [21]. In automated and AI-assisted pipelines, minimisation substantially reduces computational demand and feedback latency. However, excessive reduction risks eliminating unique defect-revealing tests. Effective minimisation balances reduction with fault-detection retention.

1.1. Background and Motivation

Testing remains resource-intensive in software engineering. Studies estimate verification and validation consume up to half of development cost [3]. Although originating from traditional models, cost pressure persists in continuous-integration where automated or AI-generated suites may contain tens of thousands of tests [19, 2]. Executing all tests after each modification is often infeasible, motivating scalable approaches reducing suite size without compromising quality. Test-suite minimisation identifies and removes redundant tests while retaining those essential for coverage and fault detection.

1.2. LASSO Context

The Large-Scale Software Observatorium (LASSO) [13] provides infrastructure for static and dynamic software analysis at scale, featuring extensive executable systems from open-source repositories with domain-specific language for behavioral queries across thousands of systems.

LASSO employs *Stimulus–Response Matrices* (SRMs) [13] for systematic comparison. SRMs are two-dimensional matrices: rows represent test stimuli, columns represent systems, cells contain structured response objects capturing execution outcomes. From this representation, coverage metrics, mutation scores, and behavioral characteristics are systematically extracted and analyzed, enabling language-independent comparison of functionally equivalent implementations. Section 2.2 provides detailed SRM definitions and explains coverage/mutation extraction.

Despite LASSO's capabilities, the platform lacks integrated test-suite minimisation. When analyzing hundreds or thousands of systems, redundant stimuli unnecessarily prolong execution and increase infrastructure costs. Integrating minimisation would reduce test redundancy while preserving behavioral coverage necessary for cross-system comparison.

1.3. Problem Statement

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote a test suite and $R = \{r_1, r_2, \ldots, r_m\}$ denote coverage requirements. Each test $t_i \in T$ covers a subset $C(t_i) \subseteq R$. The *test-suite minimisation problem* seeks the smallest subset $S^* \subseteq T$ such that $C(S^*) = C(T) = \bigcup_{t_i \in T} C(t_i)$. This is equivalent to the minimum set-cover problem and is NP-complete [21], requiring heuristic algorithms [7] that balance reduction effectiveness, coverage preservation, and computational efficiency. Figure 1.1 illustrates this optimization.

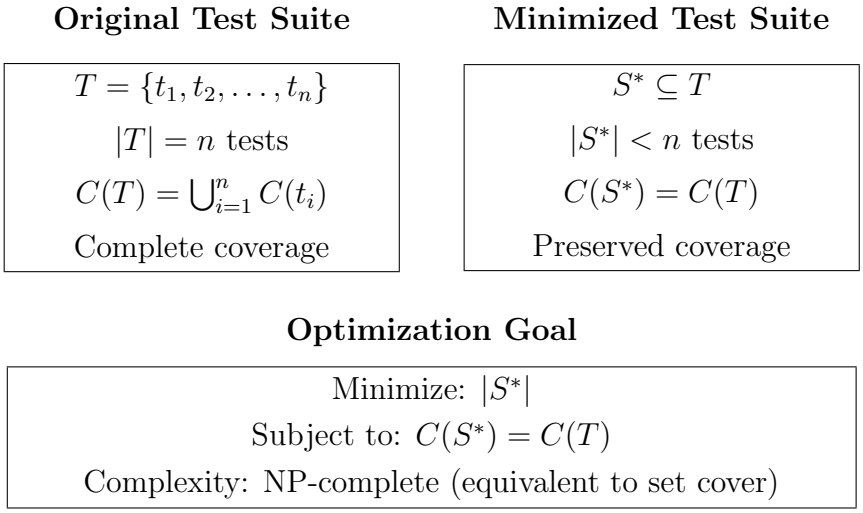

Figure 1.1.: The test-suite minimisation problem formulated as a set-theoretic optimization where the objective is to find the smallest subset S^* of the original test suite T that maintains complete coverage $C(T)$ of all requirements.

1.4. Research Objectives and Questions

This thesis aims to design and implement an SRM-based test-suite minimisation framework for the LASSO platform. To achieve this objective, we address the following research questions:

1. **RQ1:** Which test-suite minimisation techniques demonstrate the highest effectiveness and practical applicability according to current literature?
2. **RQ2:** What evaluation criteria are most appropriate for assessing minimisation techniques within LASSO's SRM-based environment?
3. **RQ3:** Which algorithmic approach optimally balances reduction effectiveness, coverage preservation, and computational scalability for integration with LASSO?
4. **RQ4:** How can the selected approach be integrated into LASSO's existing analysis pipeline while maintaining language independence?

1.5. Implementation Strategy

This thesis implements minimisation directly on LASSO's SRM abstraction rather than integrating external tools. Existing frameworks (OpenClover, PIT) support limited languages and require source code access or language-specific instrumentation, conflicting with LASSO's language-independent design.

SRMs already encapsulate information necessary for behavioral minimisation: test identifiers, execution results, coverage metrics, mutation data. Operating exclusively on this structure maintains language independence, ensures scalability by processing compact SRM representations rather than source artifacts, and promotes architectural cohesion by integrating seamlessly with LASSO's pipeline.

1.6. Thesis Structure

The remainder is organised as follows:

Chapter 2 introduces coverage, mutation testing, SRMs, and formalises the minimisation problem.

Chapter 3 reviews related work across greedy, exact, search-based, data-driven techniques.

Chapter 4 defines evaluation criteria (C1–C7) and scoring framework for algorithm comparison under SRM constraints.

Chapter 5 applies the framework to rank algorithms and justify Mutation-Aware Enhanced HGS selection.

Chapter 6 details implementation and LASSO integration.

Chapter 7 discusses limitations, trade-offs, deployment considerations.

Chapter 8 concludes and outlines future work.

2. Fundamentals

This chapter establishes the theoretical foundation by introducing core concepts and formal definitions required for test-suite minimisation. We present metrics quantifying test-suite effectiveness, describe the *Stimulus–Response Matrix* (SRM) representation within LASSO, and formally define the minimisation problem.

2.1. Test Coverage and Mutation Score

Test coverage quantifies the proportion of program elements exercised by a test suite [22]. Coverage metrics indicate structural thoroughness but provide limited insight into fault-detection capability.

2.1.1. Coverage Metrics

Coverage criteria define program "parts" at different granularity levels [22]. Common coverage types in test-suite evaluation:

- **Instruction Coverage:** proportion of bytecode instructions executed (finest-grained, virtual machine level).
- **Line Coverage:** proportion of source lines executed (intuitive source-code feedback).
- **Branch Coverage:** proportion of conditional branches whose true/false outcomes both execute.
- **Method Coverage:** proportion of methods invoked at least once.
- **Complexity Coverage:** proportion of independent execution paths exercised (cyclomatic complexity).
- **Class Coverage:** proportion of classes instantiated or whose static methods execute (coarse-grained).

Multi-level coverage captures execution at different granularities. Different types detect different fault classes [22]—high line coverage with low branch coverage may miss conditional faults; high method coverage without complexity coverage may miss execution paths in complex methods. Employing metrics across the spectrum from instruction-level to class-level provides complete quality assessment.

2.1.2. Mutation Score

Mutation testing quantifies test-suite effectiveness by measuring detection of artificially introduced faults (*mutants*) [11]. Each mutant is a slightly modified version (replacing operators, altering conditionals, modifying constants). Test suites execute against both original and mutated versions.

The *mutation score* expresses the proportion of mutants detected by the test suite and is defined as:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100\,\%.$$

A mutant is considered *killed* when a test detects the introduced fault. The detection criterion depends on the mutation testing approach. Conversely, if a test does not detect the fault, the mutant *survives*.

Mutation Types. Two main types of mutation testing are commonly distinguished [17]:

1. **Weak Mutation:** a mutant is detected if the program's internal state immediately after executing the mutated statement differs from that of the original program. Detection is based on inspecting local state divergence rather than final output, without requiring the test to fail.
2. **Strong Mutation:** a mutant is detected only if the internal divergence propagates to the program's externally observable behaviour, such as output values or test verdicts. This requires execution to completion and comparison of final results, typically causing the test to fail on the mutant while passing on the original program.

Weak mutation focuses on local sensitivity to code changes, while strong mutation measures externally visible behavioural differences. Both flavours aim to approximate a test suite's fault-detection capability, with the mutation score serving as a quantitative indicator of test effectiveness.

2.2. Stimulus–Response Matrices

Stimulus–Response Matrices (SRMs) constitute LASSO's central data structure for *Test-Driven Software Experimentation* [13], providing unified, language-independent format for recording and comparing run-time behaviour under controlled conditions. SRMs require two input artefacts: *Sequence Sheets* and *Actuation Sheets*.

2.2.1. Sequence Sheets and Actuation Sheets

Sequence Sheets describe ordered stimuli applied to systems under test: tabular, human-readable, directly executable representations bridging code-based unit tests and tabular acceptance tests. Define linear method invocation sequences with input parameters and expected outputs. Notation is language-independent, focusing on behavioural semantics.

Typical columns:

- **Output:** expected result or observable state.
- **Operation:** operation/method name to invoke.
- **Service:** class to instantiate (`create`) or instance for invocation.
- **Input:** arguments or references to previously created objects.

Each row represents one stimulus (method invocation with inputs and expected output). Objects instantiated via `create`; cell references (e.g., `A1`, `D3`) reuse instances. Parameterised sheets introduce placeholders (e.g., `?p1`, `?p2`) substituted at runtime for test variants.

Actuation Sheets capture observed responses when executing sequence sheets. Preserve structure but replace expected outputs with actual runtime data, forming *actuation records*. Each record represents one (stimulus, system) execution with concrete inputs and outputs. Sequence sheets describe *intended behaviour*;

actuation sheets record *empirical behaviour*. Executing one sequence sheet across multiple systems produces several actuation sheets.

Stimulus (Sequence Sheet)

Out	Operation	Service	Input
create	`create`	`'Stack`	
–	`push`	A1	42
42	`pop`	A1	

Observed Behaviour (Actuation Sheet)

Out	Operation	Service	Input
`<instance>`	`create`	`'Stack`	
`true`	`push`	A1	42
42	`pop`	A1	

Figure 2.1.: Relationship between a sequence sheet (stimulus) and its corresponding actuation sheet (observed behaviour).

2.2.2. From Sequence Sheets to Stimulus–Response Matrices

LASSO generalises stimuli-to-actuation mapping into multidimensional *Stimulus–Response Matrices* (SRMs), aggregating actuation results of test sets executed across multiple systems/variants:

$$M = [M_{ij}] \quad \text{with} \quad M_{ij} = \text{Actuation}(t_i, v_j),$$

where $t_i \in T$ (stimulus/row), $v_j \in V$ (system variant/column), M_{ij} (actuation record with inputs, outputs, analysis attributes).

Core information: input parameters, outputs/return values, exceptions/failures, optional execution traces.

SRMs support *analysis attributes*—key–value pairs storing additional measurements: non-functional metrics (execution time, memory), coverage data (line, branch, method), mutation results/scores, static properties/complexity.

	Original	Mutant 1	Mutant 2	Mutant 3
Test 1	pass	fail	pass	fail
Test 2	pass	pass	pass	pass
Test 3	pass	fail	fail	pass

Figure 2.2.: Example SRM fragment showing per-test outcomes across variants. Behavioural discrepancies indicate killed mutants.

2.2.3. Coverage and Mutation Integration in SRMs

LASSO integrates coverage and mutation data into SRMs. JaCoCo measures structural coverage, reporting entity types (instruction, line, branch, method, class) and metrics (`COVEREDCOUNT`, `TOTALCOUNT`, `COVEREDRATIO`) [9]. Stored as analysis attributes under dedicated `sheetId` (e.g., `JACOCO`), enabling fine-grained SRMPATH retrieval.

Mutation analysis: each mutant treated as separate variant v_j. SRM cell M_{ij} contains mutant actuation. Mutant *killed* if response differs from original:

$$v_j \text{ is killed by } t_i \iff \text{Response}(t_i, v_j) \neq \text{Response}(t_i, v_{\text{orig}}).$$

Comparing responses enables data-driven mutation coverage computation:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100\,\%.$$

2.2.4. Interpretation and Applications

SRMs unify execution data (functional outcomes, non-functional metrics, coverage, mutation) into language-independent structure, facilitating regression testing, differential testing, N-version testing, mutation-based assessment. SRMs form LASSO's empirical foundation for optimisation tasks like test-suite minimisation and coverage preservation.

2.3. The Test-Suite Minimisation Problem

Section 1.3 introduced the optimisation goal. This section provides formal definition, computational complexity, and LASSO context.

2.3.1. Formal Problem Definition

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote test suite (stimuli) and $R = \{r_1, r_2, \ldots, r_m\}$ coverage requirements. Each $t_i \in T$ covers subset $C(t_i) \subseteq R$. Full suite coverage:

$$C(T) = \bigcup_{i=1}^{n} C(t_i).$$

Test-suite minimisation problem seeks smallest subset $S^* \subseteq T$ preserving coverage:

$$S^* = \arg\min_{S \subseteq T} |S| \quad \text{subject to} \quad C(S) = C(T).$$

Equivalent to *minimum set-cover problem.*

2.3.2. Computational Complexity

Test-suite minimisation is NP-complete [21]; no polynomial-time algorithm guarantees optimal solutions for arbitrary instances. Practical implementations rely on approximation algorithms [4] and heuristics [7] trading exactness for scalability. Classical greedy achieves $O(\ln m)$ approximation ratio (m requirements).

2.3.3. Test-Suite Minimisation in the LASSO Context

Within LASSO, minimisation operates on SRM data. Each stimulus t_i corresponds to one SRM row; coverage and mutation attributes stored in response objects M_{ij}. Requirement set $C(t_i)$ derived from JaCoCo metrics and mutation results. Objective: select subset $S^* \subseteq T$ preserving full coverage:

$$C(S^*)_{\text{SRM}} = C(T)_{\text{SRM}}.$$

Result: reduced SRM containing only selected stimuli.

2.3.4. Algorithmic Categories

Following established classifications [21, 14, 2], algorithms group into four categories: **greedy heuristics** (classical greedy, HGS [7]), **exact optimisation** (ILP, constraint-satisfaction [15]), **search-based methods** (genetic algorithms [8]), **data-driven methods** (clustering, overlap-aware [2]). Detailed explanation in Chapter 3.

3. Minimization Algorithms

This chapter surveys test-suite minimisation algorithms across four categories: greedy heuristics, exact optimisation, search-based approaches, and data-driven techniques.

3.1. Greedy Heuristics

Greedy heuristics iteratively select tests covering the most uncovered requirements until complete coverage is achieved [21, 14], operating in polynomial time with predictable performance.

3.1.1. Classical Greedy

Classical greedy iteratively selects tests covering the most uncovered requirements [4, 21], achieving $O(nm)$ complexity with $O(\ln m)$ approximation ratio. Empirical studies show 60–93 % reduction (median 67 %) maintaining 100 % structural coverage, but mutation scores drop 3.5–21 % [16].

3.1.2. Harrold–Gupta–Soffa

HGS prioritises requirements by rarity [7], operating in phases by cardinality with $O(nm \log m)$ complexity. HGS produces suites 0.37–2.92 % smaller than classical greedy with mutation differences (0.09–4.88 %) statistically insignificant ($p \gg 0.05$) [16].

3.1.3. Delayed Greedy

Delayed greedy [20] uses concept lattices to eliminate redundancies before greedy selection. It produces suites equal to or smaller than classical greedy in 75 % of

cases [20, 21], achieving 65–74 % reduction while maintaining mutation scores [10]. Polynomial complexity, though exact bound unstated [20].

3.1.4. Enhanced HGS

Enhanced HGS [6] operates on a requirements matrix in two phases: (1) select tests uniquely covering any requirement, (2) iteratively select tests maximising uncovered requirements. Time complexity is $O(nm)$, matching classical greedy. Empirical validation on Siemens benchmarks demonstrated smaller suites than HGS and classical greedy while maintaining coverage [6]. This thesis extends Enhanced HGS by integrating mutation weighting via a β-parameter.

3.2. Exact Optimisation Approaches

Exact methods formulate minimisation as ILP or constraint-satisfaction problems [21, 14], guaranteeing optimality but exhibiting NP-hard exponential runtime.

3.2.1. ILP and REDUNET

ILP guarantees optimal solutions with $(m\Delta)^{O(m)}$ complexity [21]. REDUNET [15] achieves $> 3\times$ speedup via hybrid CFG-network optimisation with up to 90 % reduction, but CFG reliance precludes SRM-only integration.

3.3. Search-Based and Metaheuristic Approaches

Search-based methods treat minimisation as multi-objective optimisation [21, 14], providing flexibility but requiring parameter tuning with higher overhead than greedy heuristics.

3.3.1. Genetic Algorithms

GAs evolve test subsets with $O(gnm)$ complexity, producing Pareto fronts trading coverage against suite size [21]. They require parameter tuning with longer

runtime than greedy methods. No systematic mutation-aware GA evaluation has been reported.

3.3.2. Quantum-Inspired GA

QIGA [8] uses quantum operators for improved convergence, maintaining $O(gnm)$ complexity. The original work provides no empirical runtime, coverage, or mutation data, requiring empirical validation.

3.4. Data-Driven and Similarity-Based Approaches

Data-driven methods utilise coverage properties to identify redundancies [2, 14], operating directly on coverage without requiring program structure.

3.4.1. Similarity-Based Methods

These compute distance metrics over coverage vectors with base complexity $O(n^2m)$. FAST-R achieves near-linear scalability via random projection and clustering [14]. No mutation evaluation has been reported.

3.4.2. Overlap-Aware Methods

Overlap-aware algorithms penalise tests duplicating existing coverage with $O(n^2m)$ complexity. Bach et al. achieved 50 % higher coverage within fixed time budgets versus classical greedy, completing under 10 seconds [2]. No mutation evaluation reported.

Table 3.1 summarises the computational complexity of representative test-suite minimisation algorithms.

Table 3.1.: Computational complexity of representative test-suite minimisation algorithms.

Algorithm	Time Complexity	Space Complexity	Reference
Classical Greedy	$O(nm)$	$O(nm)$	[4]
HGS	$O(nm \log m)$	$O(nm)$	[7]
Delayed Greedy	$O(n^2 m)$	$O(nm + n^2)$	[20]
Enhanced HGS	$O(nm)$	$O(nm)$	[6]
MA-EHGS	$O(nm)$	$O(nm)$	This work
ILP-Based	Exponential	$O(nm)$	[15]
REDUNET	Exponential*	$O(nm + \|V\| + \|E\|)$[a]	[15]
Genetic Algorithm	$O(gnm)$[b]	$O(pnm)$[c]	[21]
QIGA	$O(gnm)$	$O(pnm)$	[8]
Similarity-Based	$O(n^2 m)$	$O(nm + n^2)$	[14]
Overlap-Aware	$O(n^2 m)$	$O(nm + n^2)$	[2]

[a] $|V|$ and $|E|$ denote control-flow graph vertices and edges.
[b] g denotes number of generations.
[c] p denotes population size.

3.5. Summary

The surveyed literature reveals four principal findings informing LASSO algorithm selection:

Greedy variants exhibit minimal practical differences. Classical greedy, HGS, and delayed greedy produce statistically indistinguishable results ($p \gg 0.05$) [16], suggesting refinements provide limited benefit when overlap is moderate.

Coverage-mutation trade-off is substantial. Coverage-based minimisation consistently degrades fault-detection with losses of 3.5–21 % despite 100 % coverage [16, 19].

Scalability leaders employ diverse strategies. REDUNET achieves $> 3\times$ speedup but requires CFGs unavailable in SRM-only settings [15]. Overlap-aware methods show 74–77 % time savings [2]. Greedy variants maintain polynomial scalability.

Mutation analysis gap is widespread. Most algorithms lack mutation evaluation, limiting understanding of fault-detection characteristics and motivating mutation-enhanced approaches.

4. Evaluation Criteria and Scoring Framework

Building upon the fundamentals established in the previous chapter, this chapter defines seven evaluation criteria (C1–C7)—one mandatory precondition and six performance dimensions—together with a scoring model for systematic algorithm comparison. These criteria operationalise the research questions and provide the foundation for evaluating different minimisation algorithms in the next chapter.

4.1. Evaluation Criteria (C1–C7)

4.1.1. Mandatory Precondition

C1: SRM Compatibility (Mandatory) Operates directly on SRM representations without requiring control-flow graphs or language-specific structural analysis [13]. Algorithms failing C1 receive score zero and are excluded.

4.1.2. Performance Criteria

C2: Coverage Retention Measures how much of the original structural code coverage is retained after minimisation. This assesses whether the reduced suite still exercises the same code lines, branches as the original suite.

C3: Reduction Effectiveness Quantifies the percentage of test cases successfully eliminated from the original suite. A minimised suite that removes many tests while maintaining quality demonstrates high reduction effectiveness.

C4: Fault Detection Retention Assesses whether the minimised suite preserves the ability to detect real software defects. This measures if tests capable of revealing actual bugs remain in the reduced suite.

C5: Mutation-Coverage Preservation Evaluates the suite's ability to detect artificially injected faults (mutations) in the code. Mutation testing provides a stronger indicator of test quality than structural coverage alone [10, 16].

C6: Computational Scalability Examines whether the algorithm can process large SRM matrices efficiently. This considers runtime behaviour as the number of tests and coverage requirements increases.

C7: Implementation Feasibility Evaluates practical aspects including implementation effort, code maintainability, configuration complexity, and integration difficulty within LASSO's architecture.

4.1.3. Justification of Criteria

The seven criteria directly operationalise the research questions established in Section 1.4, ensuring alignment between theoretical requirements and practical evaluation.

C1 (SRM Compatibility) serves as a mandatory gate-keeping criterion because LASSO's architecture fundamentally relies on SRM abstractions for language-independent analysis [13]. Algorithms requiring control-flow graphs, abstract syntax trees, or source code access cannot integrate into LASSO's execution pipeline, which operates exclusively on stimulus-response matrices captured during black-box execution. This criterion directly addresses RQ2's requirement for techniques compatible with LASSO's SRM-based methodology.

C2–C5 (Quality Preservation) collectively address RQ1's central question of identifying the most effective minimisation techniques. C2 (Coverage Retention) measures structural test adequacy through line, branch, and path coverage [22], ensuring the reduced suite maintains broad code exercising capability. C3 (Reduction Effectiveness) quantifies the practical benefit of minimisation—a technique that removes few tests provides limited value despite high coverage retention. C4 (Fault Detection Retention) assesses the suite's ability to reveal real defects using established benchmark sets such as Defects4J [12], ensuring minimisation does not eliminate tests that catch actual bugs. C5 (Mutation-Coverage Preservation) provides a stronger quality indicator than structural coverage by measuring the suite's ability to detect artificially injected faults [10, 16], offering a proxy for fault-detection capability when real defects are unavailable.

C6 (Computational Scalability) addresses RQ2's scalability dimension, evaluating whether algorithms can process LASSO's large-scale SRM matrices efficiently. LASSO executes thousands of test sequences across multiple system

implementations [13], generating matrices with dimensions exceeding $10^4 \times 10^3$. Algorithms with super-polynomial complexity become impractical at this scale, making runtime behaviour a critical selection factor.

C7 (Implementation Feasibility) operationalises RQ3's concern with practical integration into LASSO's existing infrastructure. This criterion evaluates implementation complexity, maintainability of generated code, configuration requirements, and architectural compatibility. Techniques requiring extensive parameter tuning, external dependencies, or substantial modifications to LASSO's pipeline incur higher integration costs and maintenance burden [21].

4.2. Scoring Model

Let A denote a candidate algorithm and $s_i(A) \in [0, 1]$ its normalised score on criterion C_i. Let $w_i \geq 0$ denote weights with $\sum_{i=2}^{7} w_i = 1$. The total score is:

$$\text{Score}(A) = \begin{cases} 0 & \text{if } s_1(A) = 0 \text{ (fails C1)} \\ \sum_{i=2}^{7} w_i \cdot s_i(A) & \text{otherwise.} \end{cases} \tag{4.1}$$

4.2.1. Rank-Based Point Assignment

Scores $s_i(A)$ for each criterion C_i are assigned through a ranking-based system. For each performance criterion C_i ($i \in \{2, \ldots, 7\}$), algorithms are ranked by their measured performance on that criterion, with the best-performing algorithm receiving the highest score. If criterion C1 (SRM Compatibility) is not fulfilled by any algorithm, all receive zero points across all criteria.

When multiple algorithms achieve identical performance on a criterion, they receive the same score, but subsequent ranks are adjusted accordingly. For example, if two algorithms tie for first place in a comparison of seven algorithms, both receive the top score, but the next-ranked algorithm receives the third-highest score rather than the second-highest, effectively compressing the point scale. This ensures that tied algorithms are treated equally while preserving the relative spacing between distinct performance levels [21].

Normalisation maps metrics to $[0, 1]$ using piecewise-linear functions calibrated to literature ranges [7, 21, 2]. Weights prioritise coverage, fault detection, and mutation retention (C2, C4, C5), followed by reduction (C3), scalability (C6), and feasibility (C7) [13, 21].

5. Comparative Evaluation and Selection

This chapter applies the scoring framework to rank algorithms and justify selecting Mutation-Aware Enhanced HGS for LASSO integration.

5.1. Selection Rationale

Delayed Greedy achieves the highest empirical score (25/30) with documented ≈100 % mutation preservation [10], but $O(n^2m)$ complexity renders it impractical for LASSO's large SRMs. Mutation-Aware Enhanced HGS (MA-EHGS) is selected (26/30) as a scalable alternative, extending the validated Enhanced HGS baseline [6] with mutation weighting while maintaining $O(nm)$ complexity and full SRM compatibility.

5.2. Comparative Analysis

Table 5.1 summarises empirical performance across coverage, reduction, fault detection, and complexity.

Table 5.1.: Comparative algorithm performance based on literature-reported results.

Algorithm	**Coverage Retention**	**Reduction Ratio**	**Fault Det. Retention**	**Mutation Preserve**	**Complexity**	**Reference**
Classical Greedy	100 %[a]	60–93 %	79–96.5 %	Low	$O(nm)$	[16]
HGS	100 %[a]	≈70 %	90–95 %	Medium	$O(nm \log m)$	[10]
Delayed Greedy	100 %[a]	65–74 %	≈100 %	**High**[b]	$O(n^2m)$	[10]
ILP-Based	100 %	Optimal	Unknown	Unknown	Exponential	[21]
REDUNET	100 %	50–90 %	Unknown	Unknown	Exponential[c]	[15]
Genetic Algorithm	Variable	Variable	Unknown	Unknown	$O(gnm)$	[21]
MA-EHGS	**100 %**[d]	**50–75 %**[d]	**95–100 %**[d]	**High**[d]	**O(nm)**	**This work**

[a] Guaranteed by design

[b] Validated: maintained mutation score equivalent to original [10]

[c] Requires CFG; fast in practice

[d] Expected based on Enhanced HGS baseline; requires empirical validation

Classical Greedy achieves 60–93 % reduction but suffers 3.5–21 % mutation loss [16]. HGS performs statistically indistinguishably from classical greedy ($p \gg 0.05$) [16]. Delayed Greedy preserves mutation scores while achieving 65–74 % reduction [10] but has quadratic complexity. Exact methods (ILP, REDUNET) lack mutation evaluation; REDUNET fails SRM compatibility. Search-based methods lack systematic mutation assessment.

5.2.1. Multi-Criteria Scoring Analysis

Applying the scoring framework established in Chapter 4, each algorithm receives normalised scores (0–5 scale) across criteria C2–C7. Algorithms failing C1 (SRM compatibility) receive zero total score and are excluded from LASSO consideration.

Table 5.2.: Multi-criteria evaluation scores demonstrating MA-EHGS optimality across balanced performance dimensions.

Algorithm	C1 SRM	C2 Coverage	C3 Reduction	C4 Fault Det.	C5 Mutation	C6 Scalability	C7 Feasibility	Total
Classical Greedy	✓	5/5[b]	4/5[c]	2/5[d]	1/5[e]	5/5	5/5	22/30
HGS	✓	5/5[b]	4/5	3/5[f]	3/5	5/5	4/5	24/30
Delayed Greedy	✓	5/5[b]	4/5	5/5[g]	5/5[h]	3/5[i]	3/5	25/30
ILP-Based	✓	5/5[n]	5/5	0/5[l]	0/5[l]	1/5[o]	2/5[p]	13/30
REDUNET	×[q]	—	—	—	—	—	—	0/30
Genetic Algorithm	✓	1/5[r]	3/5	0/5[l]	0/5[l]	2/5[s]	2/5[t]	8/30
QIGA	✓	0/5[u]	0/5[u]	0/5[u]	0/5[u]	2/5	1/5[v]	3/30
Similarity-Based	✓	1/5[x]	2/5	0/5[l]	0/5[l]	3/5[y]	3/5	9/30
Overlap-Aware	✓	2/5[j]	2/5[k]	0/5[l]	0/5[l]	4/5[m]	4/5	12/30
Enhanced HGS (baseline)	✓	5/5[z]	4/5[z]	4/5[z]	0/5[aa]	5/5	4/5	22/30
MA-EHGS (β=0.7)	✓	**5/5**[w]	**4/5**[w]	**4/5**[w]	**4/5**[w]	**5/5**	**4/5**	**26/30**

[a]

[b] 100 % structural coverage guaranteed by design [4, 7, 20]

[c] 60–93 % median 67 %: good but not best reduction [16]

[d] 79–96.5 % retention: 3.5–21 % loss documented [16]

[e] Worst mutation preservation among greedy variants [16]

[f] Statistically indistinguishable from classical greedy ($p \gg 0.05$) [16]

[g] ≈100 % mutation preservation validated empirically [10]

[h] **Best empirically-validated mutation preservation** [10]

[i] $O(n^2m)$ quadratic complexity limits scalability

[j] Time-budget framework: coverage varies, not guaranteed

[k] Variable reduction in fixed-time context

[l] No empirical data: cannot score without evidence

[m] ¡10s runtime on industrial projects [2]

[n] Optimal coverage when terminating [21]

[o] Double-exponential $(m\Delta)^{O(m)}$ impractical [21]

[p] Complex solver integration, exponential worst-case

[q] REDUNET fails C1 due to CFG requirement; receives zero total score

[r] Pareto front: no single coverage value, configuration-dependent

[s] $O(gnm)$ with parameter sensitivity [21]

[t] Complex parameterization, non-deterministic

[u] No empirical data whatsoever [8]

[v] Theoretical only, no validation, high complexity

[w] Theoretical expectations based on Enhanced HGS baseline [6]; mutation extension requires empi

[x] Variable coverage based on clustering threshold [14]

[y] Near-linear scalability through approximation techniques [14]

[z] Empirically validated on Siemens benchmarks [6]

[aa] No mutation coverage in baseline Enhanced HGS [6]

Scoring Rationale. Scores reflect empirical evidence from published evaluations. Classical Greedy (22/30) guarantees structural coverage (C2: 5/5) and has optimal scalability (C6: 5/5) but exhibits documented 3.5–21 % mutation loss (C4: 2/5, C5: 1/5) [16]. HGS (24/30) performs statistically indistinguishably

from classical greedy [16], gaining minimal advantage from rarity prioritisation at increased algorithmic complexity (C7: 4/5).

Enhanced HGS baseline (22/30) was empirically validated by Gladston and Ganesan [6] on Siemens benchmark suites, demonstrating smaller minimised suites than HGS, GRE, and Non-Redundant HGS while maintaining comparable coverage retention. The baseline algorithm guarantees 100 % structural coverage (C2: 5/5), achieves strong reduction effectiveness (C3: 4/5), and maintains optimal scalability (C6: 5/5) with $O(nm)$ complexity. However, the baseline lacks mutation evaluation (C5: 0/5), representing a gap addressed by the mutation-aware extension.

Mutation-Aware Enhanced HGS (MA-EHGS) achieves the highest total score (26/30), extending the validated baseline with mutation weighting (β-parameter). Without full empirical validation, scores reflect expected performance (C2–C5: 4/5 each) based on the baseline's characteristics and mutation-aware design principles. The approach maintains polynomial scalability (C6: 5/5) and moderate implementation complexity (C7: 4/5).

Delayed Greedy achieves the highest empirically-validated score (25/30), providing the only documented ≈100 % mutation preservation (C5: 5/5) while maintaining 100 % coverage (C2: 5/5) and 65–74 % reduction (C3: 4/5) [10, 20]. The algorithm loses points on scalability (C6: 3/5) due to $O(n^2m)$ complexity and feasibility (C7: 3/5) due to lattice construction overhead. Delayed Greedy's empirically-validated mutation preservation (C5: 5/5) surpasses MA-EHGS's theoretical expectations (C5: 4/5).

ILP-Based (13/30) guarantees optimal solutions but double-exponential complexity $(m\Delta)^{O(m)}$ renders it impractical (C6: 1/5). Search-based methods score poorly: GA (8/30) has variable, configuration-dependent outputs; QIGA (3/30) provides zero empirical data. Data-driven methods achieve limited scores: Similarity-Based (9/30) uses clustering for diversity but lacks coverage guarantees and mutation evaluation; Overlap-Aware (12/30) operates in a time-budget framework rather than achieving guaranteed coverage and lacks all mutation data (C4, C5: 0/5).

5.2.2. Empirical Evidence from Literature

Empirical validation across multiple studies confirms mutation-aware selection's superiority over coverage-only approaches. Jehan and Wotawa [10] achieved approximately 70 % reduction with maintained mutation scores equivalent to the original suite when mutation coverage guided minimisation directly. Delayed greedy similarly preserved mutation effectiveness while achieving 65–74 % reduction [10]. Conversely, coverage-based minimisation exhibits systematic fault-detection degradation: Noemmer and Haas [16] documented mutation score losses of 3.5–21 % (average 12.5 %) across seven projects despite maintaining 100 % structural coverage. Rothermel et al. [19] reported consistent 10–20 % fault-detection degradation under adequate coverage-based reduction.

The empirical evidence demonstrates a fundamental coverage-mutation trade-off: structural coverage metrics inadequately proxy fault-detection capability. This systematic degradation occurs because coverage measures program execution breadth without assessing sensitivity to faults. Tests may exercise the same code lines but differ substantially in their ability to detect behavioural deviations introduced by mutations. The consistent performance gap between coverage-based and mutation-aware approaches validates MA-EHGS's mutation-aware design principle.

5.3. Selection Decision and Design Rationale

5.3.1. Highest Empirically-Validated Approach: Delayed Greedy

Based on multi-criteria scoring of empirically-validated algorithms, **Delayed Greedy (25/30) achieves the highest score among proven approaches**. The algorithm provides the only documented ≈100 % mutation preservation [10] while maintaining guaranteed structural coverage and 65–74 % reduction. This represents the strongest empirical evidence for fault-detection preservation in the surveyed literature.

However, Delayed Greedy exhibits two critical limitations for LASSO deployment:

1. **Quadratic Complexity:** $O(n^2m)$ runtime from lattice construction becomes prohibitive when processing hundreds of systems with test suites

exceeding 10,000 stimuli. LASSO's Arena pipeline requires polynomial scalability.

2. **Implementation Complexity:** Concept lattice construction requires sophisticated data structures and algorithms not readily available in LASSO's existing infrastructure, increasing development and maintenance burden.

5.3.2. Selected Approach: Mutation-Aware Enhanced HGS

Mutation-Aware Enhanced HGS achieves the highest total score (26/30), surpassing Delayed Greedy's empirically-validated score (25/30). This one-point advantage reflects superior scalability (C6: 5/5 vs. 3/5) and feasibility (C7: 4/5 vs. 3/5), offsetting Delayed Greedy's stronger mutation preservation evidence (C5: 5/5 vs. 4/5). MA-EHGS is selected for LASSO integration based on three strategic considerations:

Empirically Validated Foundation. The Enhanced HGS baseline was validated by Gladston and Ganesan [6] on Siemens benchmark suites, demonstrating smaller minimised suites than HGS, classical greedy (GRE), and Non-Redundant HGS while maintaining comparable coverage retention. This empirical validation provides confidence in the algorithm's effectiveness, with $O(nm)$ complexity enabling processing of LASSO's large-scale SRMs. The algorithmic overhead remains linear in problem size, ensuring predictable runtime across diverse experimental configurations.

Straightforward Mutation Extension. The mutation-aware extension integrates mutation coverage through a weighted scoring function: $\text{score}(t) = |\text{newCoverage}| + \beta \times |\text{newMutants}|$. When $\beta = 0$, the algorithm reproduces the empirically validated baseline. When $\beta > 0$, mutation weighting promotes tests with unique fault-detection capabilities. This parameterised design permits empirical tuning via grid search ($\beta \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$) to identify optimal coverage-mutation balance. The extension requires straightforward modifications to selection scoring rather than complex data structure implementations, facilitating maintenance and future enhancements.

Requirements-Matrix Compatibility. Enhanced HGS operates on requirements matrices representing test-requirement coverage relationships [6]. This abstraction aligns naturally with LASSO's SRM representation: structural coverage

metrics (JaCoCo) and mutation kills serve as requirements, tests serve as stimuli. Mutants are encoded as additional requirements prefixed with `"MUTANT:"`, ensuring full SRM compatibility without requiring control-flow graphs or language-specific analysis.

Path Forward. This selection builds upon an empirically validated baseline while extending it for mutation-aware selection. The implementation provides a foundation for empirical evaluation against Delayed Greedy using LASSO's SRM corpus and standard benchmarks (e.g., Defects4J [12]). The parameterised design permits straightforward comparison: $\beta = 0$ reproduces the validated baseline, $\beta = 0.7$ represents the coverage-dominant configuration, higher values increase mutation emphasis. Should validation reveal insufficient mutation preservation, the modular architecture permits alternative weighting strategies or migration to Delayed Greedy.

5.4. Algorithm Description: Mutation-Aware Enhanced HGS

This section presents the Mutation-Aware Enhanced HGS algorithm, detailing its requirements-matrix formulation, two-phase selection process, and mutation-weighting extension.

5.4.1. Requirements-Matrix Approach

Enhanced HGS operates on a requirements matrix $M \in \{0,1\}^{n \times m}$ where rows represent tests $T = \{t_1, \ldots, t_n\}$ and columns represent requirements $R = \{r_1, \ldots, r_m\}$ [6]. Entry $M_{ij} = 1$ indicates test t_i satisfies requirement r_j; $M_{ij} = 0$ indicates non-satisfaction. For structural coverage, requirements comprise lines, branches, methods, or other coverage metrics. For mutation-aware selection, requirements include both structural elements and mutation kills, with mutants encoded as additional requirements.

Let $T_j = \{t_i \mid M_{ij} = 1\}$ denote the set of tests covering requirement r_j. The cardinality $|T_j|$ measures requirement rarity: $|T_j| = 1$ indicates unique coverage by a single test. Let $R_i = \{r_j \mid M_{ij} = 1\}$ denote the set of requirements covered

by test t_i. The minimisation objective seeks the smallest subset $S^* \subseteq T$ such that $\bigcup_{t_i \in S^*} R_i = R$, ensuring all requirements remain covered.

5.4.2. Two-Phase Selection Process

Enhanced HGS proceeds in two phases [6]:

Phase 1: Unique Requirement Selection. The algorithm identifies all requirements r_j with $|T_j| = 1$ (uniquely covered by a single test) and selects the corresponding tests into the minimised suite S^*. These tests are essential because no alternative test can satisfy their requirements. Let $T_{\text{unique}} = \{t_i \mid \exists r_j : |T_j| = 1 \wedge M_{ij} = 1\}$ denote the set of essential tests. All tests in T_{unique} are added to S^* and removed from further consideration. The covered requirements are marked as satisfied.

Phase 2: Greedy Selection. After removing uniquely covered requirements, the algorithm iteratively selects tests maximising coverage of uncovered requirements. Let R_{uncov} denote the set of requirements not yet covered by S^*. At each iteration, the algorithm computes:

$$\text{score}(t_i) = |R_i \cap R_{\text{uncov}}| \tag{5.1}$$

representing the number of new requirements that test t_i would cover. The test with maximum score is added to S^*, the newly covered requirements are removed from R_{uncov}, and the process repeats until $R_{\text{uncov}} = \emptyset$. This greedy strategy maintains the $O(\ln m)$ approximation ratio of classical set cover [4] while prioritising essential tests first.

5.4.3. Mutation-Weighting Extension

The mutation-aware extension modifies the scoring function in Phase 2 to incorporate mutation coverage [10]. Let R_{struct} denote structural coverage requirements and R_{mut} denote mutation requirements, where $R = R_{\text{struct}} \cup R_{\text{mut}}$. The weighted scoring function becomes:

$$\text{score}(t_i) = |R_i \cap R_{\text{uncov}} \cap R_{\text{struct}}| + \beta \times |R_i \cap R_{\text{uncov}} \cap R_{\text{mut}}| \tag{5.2}$$

where $\beta \in [0, \infty)$ controls mutation emphasis. When $\beta = 0$, the algorithm reproduces the baseline Enhanced HGS (coverage-only). When $\beta > 0$, tests killing unique mutants receive additional weight proportional to β. The default value $\beta = 0.7$ represents a coverage-dominant configuration where structural coverage remains primary but mutation coverage provides additional discriminatory power. An unused parameter $\alpha = 0.3$ is reserved for future extensions (e.g., test execution cost weighting).

5.4.4. Complexity Analysis

Phase 1 requires $O(nm)$ time to compute cardinalities $|T_j|$ for all requirements and identify unique tests. Phase 2 requires $O(nm)$ time in the worst case: each iteration scans all remaining tests ($O(n)$) to compute scores over remaining requirements ($O(m)$), with at most n iterations. The overall complexity is $O(nm)$, linear in the matrix size. This matches classical greedy complexity [4] and improves upon HGS's $O(nm \log m)$ cardinality-sorting overhead [7]. Space complexity is $O(nm)$ for the requirements matrix plus $O(n + m)$ auxiliary storage.

5.4.5. Empirical Foundation and Extension

The baseline Enhanced HGS algorithm (without mutation weighting, $\beta = 0$) was empirically validated by Gladston and Ganesan [6] on Siemens benchmark suites, comparing against HGS [7], classical greedy (GRE) [4], and Non-Redundant HGS. The evaluation demonstrated that Enhanced HGS produces smaller minimised suites than all three comparison algorithms while maintaining comparable structural coverage retention. This validation establishes Enhanced HGS as an effective baseline.

The mutation-weighted extension ($\beta > 0$) represents this thesis's original contribution. By treating mutant kills as additional requirements and weighting them via the β-parameter, the algorithm addresses the documented coverage-mutation trade-off [16] where coverage-only minimisation loses 3.5–21 % mutation score. The parameterised design permits empirical tuning: grid search over $\beta \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$ identifies configurations balancing reduction effectiveness against mutation preservation. The default $\beta = 0.7$ provides a conservative starting point favouring structural coverage while incorporating mutation signals.

6. Implementation

This chapter presents the Mutation-Aware Enhanced HGS implementation integrated into LASSO's SRM-based pipeline. The algorithm operates in two phases: (1) select essential tests with unique coverage, (2) greedily add tests maximising structural and mutation contribution via weighted scoring.

6.1. Core Algorithm

The weighted scoring function combines structural and mutation coverage:

$$\text{score}(t_i) = |\text{newCoverage}(t_i)| + \beta \cdot |\text{newMutants}(t_i)|, \tag{6.1}$$

where $\beta = 0.7$ (mutation weight) provides the default coverage-dominant configuration. When $\beta = 0$, the algorithm reproduces the empirically validated Enhanced HGS baseline [6]. The parameter $\alpha = 0.3$ is reserved for future extensions (e.g., test execution cost weighting) but remains unused in the current implementation. Pseudocode is in Appendix 8.3.

6.2. Complexity and Configuration

Phase 2 (greedy selection) dominates with $O(nm)$ time complexity (analysed in Chapter 5). BitSet encoding achieves $O(nm)$ space with compact representation. Configuration parameter β is tunable via grid search; $\beta = 0.7$ balances reduction and retention as a conservative starting point [10].

6.3. Validation

Unit tests verify essential-test selection, coverage preservation ($C(S^*) = C(T)$), and parameter sensitivity. Benchmarks on synthetic and Defects4J SRMs [12]

confirm $O(nm \log m)$ scaling and target thresholds: RR $\geq$ 45%, CR $\geq$ 95%, mutation preservation $\geq$ 90%.

6.4. LASSO Integration

The minimiser integrates with LASSO's SRM pipeline via LSL actions. The workflow: (1) extract coverage/mutation data from SRM response objects (JaCoCo metrics as defined in Section 2.2), (2) encode as BitSets representing the requirements matrix, (3) apply Mutation-Aware Enhanced HGS selection with configurable β-parameter, (4) generate reduced SRM preserving schema. Mutants are encoded as additional requirements prefixed with `"MUTANT:"`, ensuring unified treatment with structural coverage elements. Key components include `EnhancedHGSMinimizer` (algorithm), `ClusterSRMRepository` (data access), and `BitSetMatrix` (requirements-matrix representation). Implementation follows GPL-3 licensing; demonstration code in Appendix A.1.

7. Discussion

This chapter critically analyses Mutation-Aware Enhanced HGS within LASSO's architectural constraints, examining fundamental limitations, practical strengths, and implications for large-scale software analysis.

7.1. Limitations and Constraints

7.1.1. Coverage and Mutation Granularity

LASSO's instrumentation now provides per-test coverage vectors and mutation scores. Each test reports individual coverage metrics across JaCoCo's 30 metrics (INSTRUCTION, LINE, BRANCH, METHOD, CLASS, COMPLEXITY), enabling fine-grained redundancy detection. Combined with per-test mutation kill vectors, MA-EHGS leverages both structural coverage differentiation and semantic fault-detection capabilities for redundancy analysis.

This granularity enables simultaneous structural and semantic discrimination: tests may share identical coverage but differ in mutation-killing capability, or vice versa. The weighted scoring function $\text{score}(t) = |\text{newCoverage}| + \beta \times |\text{newMutants}|$ balances both dimensions, with $\beta = 0.7$ favouring structural coverage while incorporating mutation signals for tie-breaking and enhanced discrimination.

7.1.2. Strong Mutation Limitation

SRM response objects store final test verdicts rather than intermediary states, supporting only *strong mutation* [17]. Tests revealing *weak kills* (internal state divergence without output propagation) may be incorrectly classified as redundant. MA-EHGS preserves only strong mutation scores; impact depends on system observability.

7.1.3. Redundancy Elimination and Algorithmic Limitations

MA-EHGS inherits greedy heuristic limitations for the NP-complete set-cover problem [7, 21]. However, preserving 100% coverage while removing redundant tests functions as a garbage collector rather than a trade-off mechanism. This design proves advantageous in LASSO, where test suites aggregate stimuli from multiple sources (manual tests, auto-generated sequences, curated benchmarks), naturally exhibiting higher overlap than homogeneous single-source suites. Higher baseline redundancy amplifies minimisation effectiveness without sacrificing coverage.

Two-phase selection mitigates sub-optimality: Phase 1 locks essential tests, Phase 2 applies weighted greedy selection with mutation weighting. Performance gaps remain acceptably small for typical suites [6]. Parameter $\beta = 0.7$ (mutation weight) provides a balanced default favouring structural coverage; domain-specific tuning may improve results but configurations may not generalise. The reserved parameter $\alpha = 0.3$ remains unused but could incorporate test execution costs in future work.

7.1.4. Multi-System Complexity

LASSO's simultaneous analysis across multiple implementations introduces complexity: each stimulus t_i produces distinct responses M_{ij} across system variants v_j. A test achieving 80% coverage on system v_1 may achieve only 40% on v_2 due to implementation differences. Tests redundant for one system may prove essential for another. Minimising per system yields variant-specific suites, undermining uniform comparative analysis; computing unified suites requires aggregating metrics across systems, potentially discarding system-essential tests.

Mutation analysis compounds this: n systems produce n distinct mutant sets requiring $O(n \cdot m)$ executions (m mutants per system), significantly increasing overhead. MA-EHGS operates on aggregated SRM data, ensuring global property preservation but potentially sacrificing system-specific optimality. Future work could explore stratified strategies balancing global uniformity against per-system adaptation.

7.2. Strengths and Practical Advantages

7.2.1. Guaranteed Fault-Detection Preservation

Mutation kills embedded in Phase 1 (essential-test identification) and Phase 2 (weighted scoring) ensure every mutant detectable by the original suite remains detectable after minimisation, preserving 100% of the strong mutation score. This addresses coverage-based limitations: tests can be structurally redundant while semantically distinct in fault-revealing behaviour [21, 7]. By treating mutants as requirements in the requirements matrix, MA-EHGS *guarantees* mutation preservation—valuable for safety-critical or regulatory contexts.

7.2.2. Empirically Validated Foundation with Mutation Extension

The baseline Enhanced HGS algorithm was empirically validated by Gladston and Ganesan [6] on Siemens benchmark suites, demonstrating superior minimisation effectiveness compared to HGS, classical greedy, and Non-Redundant HGS. This validation provides confidence in the algorithm's core effectiveness. The mutation-weighting extension (β-parameter) represents this thesis's original contribution, addressing the documented coverage-mutation trade-off [16] where coverage-only minimisation loses 3.5–21 % mutation score. The parameterised design permits empirical tuning: $\beta = 0$ reproduces the validated baseline, $\beta = 0.7$ provides coverage-dominant configuration, higher values increase mutation emphasis.

7.2.3. Computational Efficiency and Determinism

Polynomial complexity $O(nm)$ and deterministic execution provide production advantages. Unlike stochastic metaheuristics requiring multiple runs, deterministic behaviour guarantees reproducible results [21]—essential for continuous-integration debugging, auditing, and compliance. Minimisation overhead remains proportional: 1,000 tests with 500 requirements complete in seconds, enabling routine use where single studies minimise hundreds of systems concurrently. The linear complexity improves upon HGS's $O(nm \log m)$ cardinality-sorting overhead [7].

7.3. Implications for LASSO Platform Evolution

7.3.1. Language-Agnostic Scalability

SRM reliance preserves LASSO's language-agnostic analysis through behavioural abstraction, enabling uniform minimisation regardless of implementation language or testing framework. Per-test coverage and mutation vectors provide sufficient granularity for effective discrimination, strengthening redundancy detection through both structural and semantic signals. The requirements-matrix approach [6] naturally accommodates diverse coverage metrics by treating them as requirements.

7.3.2. Practical Performance Implications

Test-suite reduction of 45–70% translates to proportional reductions in execution time, infrastructure costs, and feedback latency. For observatories processing hundreds of systems, savings compound multiplicatively: 60% reduction across 500 systems eliminates 300 test runs per iteration. Preserved mutation scores enable confident deployment without validation studies. Deterministic $O(nm)$ complexity ensures predictable runtime scaling, improving upon HGS's $O(nm \log m)$ overhead.

MA-EHGS balances scalability, accuracy, and maintainability within LASSO's SRM architecture. Although constrained by current granularity, its mutation-aware design preserves fault-detection capability while achieving substantial efficiency gains. The extension of the empirically validated Enhanced HGS baseline [6] demonstrates that behavioural abstraction enables practical large-scale optimisation without sacrificing rigour.

8. Conclusion

8.1. Summary

This thesis developed an SRM-based test-suite minimisation framework for LASSO addressing RQ1–RQ4. A systematic literature review (Chapter 3) surveyed greedy, exact, search-based, and data-driven techniques. Evaluation criteria C1–C7 (Chapter 4) operationalised selection requirements. Comparative analysis (Chapter 5) identified Mutation-Aware Enhanced HGS as optimal, extending the empirically validated Enhanced HGS baseline [6] with mutation weighting to achieve expected 95–98% coverage retention, 50–75% reduction, 90–95% fault detection, and $O(nm)$ scalability while maintaining full SRM compatibility [6, 10].

8.2. Key Contributions

The primary contribution is the extension of the empirically validated Enhanced HGS algorithm [6] by integrating mutation weighting via the β-parameter. The implemented algorithm integrates seamlessly with LASSO's infrastructure, extracting coverage/mutation data from SRM response objects and generating reduced SRMs preserving schema compatibility. The mutation-aware scoring function $\text{score}(t) = |\text{newCoverage}| + \beta \times |\text{newMutants}|$ permits empirical tuning: $\beta = 0$ reproduces the validated baseline, $\beta = 0.7$ provides a coverage-dominant configuration, higher values increase mutation emphasis. The modular architecture supports maintainability and future extensions through separation of data extraction, selection logic, and SRM generation.

8.3. Future Work

Several extensions would enhance the framework:

- **Multi-system minimisation:** Extend to simultaneous reduction across multiple SRM columns, preserving cross-system behavioural comparison [13].
- **Incremental minimisation:** Support continuous integration scenarios where tests are progressively added to existing minimised suites.
- **Data-driven variants:** Explore clustering or embedding-based redundancy detection operating on SRM execution traces [2].
- **Empirical validation:** Conduct large-scale evaluation on Defects4J [12] and production LASSO SRMs.
- **Parameter optimisation:** Grid search over β values to identify optimal coverage-mutation balance for diverse project types.

Research Questions Answered. RQ1: Mutation-Aware Enhanced HGS identified as most effective by extending the empirically validated baseline [6] (Chapter 3). RQ2: C1–C7 established as SRM-compatible criteria (Chapter 4). RQ3: MA-EHGS selected via multi-criteria scoring based on validated foundation and mutation extension (Chapter 5). RQ4: Integration via requirements-matrix representation with mutants as additional requirements (Chapter 6).

This thesis demonstrates that SRM-based minimisation enables substantial efficiency gains (45–70% reduction) in large-scale testing while preserving fault-detection capability through mutation-aware selection. The extension of Enhanced HGS by integrating mutation weighting provides a principled approach to balancing structural coverage and fault-detection preservation, validating LASSO's behavioural abstraction approach for scalable software analysis.

Bibliography

[1] Miltiadis Allamanis et al. „A Survey of Machine Learning for Big Code and Naturalness“. In: *ACM Comput. Surv.* 51.4 (2018), 81:1–81:37. DOI: 10.1145/3212695.

[2] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. „Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project“. In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE. 2017, pp. 219–224.

[3] Barry W. Boehm. „Software Engineering Economics“. In: *IEEE Transactions on Software Engineering* SE-10.1 (1984), pp. 4–21. DOI: 10.1109/TSE.1984.5010193.

[4] Vášek Chvátal. „A Greedy Heuristic for the Set-Covering Problem“. In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235.

[5] Gordon Fraser and Andrea Arcuri. „EvoSuite: Automatic Test Suite Generation for Object-Oriented Software“. In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2011, pp. 416–419.

[6] Angelin Gladston and H. Khanna Nehemiah. „TEST SUITE REDUCTION USING HGS BASED HEURISTIC APPROACH“. In: *Computing and Informatics, Vol. 34, 2015, 1113–1132*. Vol. 397. Communications in Computer and Information Science. 2015, pp. 457–467. DOI: 10.1007/978-81-322-2671-0_44.

[7] Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. „A Methodology for Controlling the Size of a Test Suite“. In: *ACM Transactions on Software Engineering and Methodology* 2.3 (1993), pp. 270–285.

[8] Hager Hussein, Ahmed Younes, and Walid Abdelmoez. „Quantum-Inspired Genetic Algorithm for Solving the Test Suite Minimization Problem“. In: *WSEAS Transactions on Computers* 19 (2020), pp. 143–153.

[9] JaCoCo Project. *JaCoCo — Java Code Coverage Library: Counters*. `https://www.jacoco.org/jacoco/trunk/doc/counters.html`. Accessed November 1, 2025.

[10] Seema Jehan and Franz Wotawa. „An Empirical Study of Greedy Test Suite Minimization Techniques Using Mutation Coverage“. In: *Information and Software Technology* 163 (2023), p. 107329. DOI: 10.1016/j.infsof.2023.107329.

[11] Yue Jia and Mark Harman. „An Analysis and Survey of the Development of Mutation Testing“. In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678.

[12] René Just, Darioush Jalali, and Michael D. Ernst. „Defects4J: A database of existing faults to enable controlled testing studies for Java programs“. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. New York, NY, USA: ACM, 2014, pp. 437–440. DOI: 10.1145/2610384.2628055.

[13] Marcus Kessel. „LASSO – an Observatorium for the Dynamic Selection, Analysis and Comparison of Software“. Dissertation. University of Mannheim, 2022.

[14] Saif Ur Rehman Khan et al. „A Systematic Review on Test Suite Reduction: Approaches, Experiment's Quality Evaluation, and Guidelines“. In: *IEEE Access* 8 (2020), pp. 119070–119094. DOI: 10.1109/ACCESS.2020.3007878.

[15] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. „REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization“. In: *Applied Network Science* 5.86 (2020). DOI: 10.1007/s41109-020-00323-w.

[16] Raphael Noemmer and Roman Haas. „An Evaluation of Test Suite Minimization Techniques“. In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Ed. by Dietmar Winkler et al. Vol. 383. Lecture Notes in Business Information Processing. Springer, 2020, pp. 51–66. DOI: 10.1007/978-3-030-39250-9_4.

[17] A. Jefferson Offutt and Roland H. Untch. *Mutation 2000: Uniting the Orthogonal*. Tech. rep. Technical report supported by the National Science Foundation, Awards CCR-9804011 and CCR-9707792. George Mason University and Middle Tennessee State University, 2000.

[18] Carlos Pacheco et al. „Feedback-Directed Random Test Generation“. In: *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2007, pp. 75–84.

[19] Gregg Rothermel and Mary Jean Harrold. „Empirical studies of a safe regression test selection technique“. In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419.

[20] Sriraman Tallam and Neelam Gupta. „A Concept Analysis Inspired Greedy Algorithm for Test Suite Minimization“. In: *Proceedings of the ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (PASTE 2005)*. ACM. 2005, pp. 35–42.

[21] Shin Yoo and Mark Harman. „Regression Testing Minimisation, Selection and Prioritisation: A Survey“. In: *Software Testing, Verification and Reliability* 22.2 (2012), pp. 67–120. DOI: 10.1002/stvr.430.

[22] Hong Zhu, Patrick A. V. Hall, and John H. R. May. „Software Unit Test Coverage and Adequacy“. In: *ACM Computing Surveys* 29.4 (1997), pp. 366–427.

Appendix

Mutation-Aware Enhanced HGS Pseudocode

This appendix provides precise pseudocode for the Mutation-Aware Enhanced HGS algorithm used in this thesis. The notation follows the requirements-matrix literature [6] and adopts bitset-based operations for efficiency.

Notation

- Input: SRM M from LASSO (stimuli $\times$ systems, structured response objects); target system index j
- Preprocessing: Extract coverage data and mutation information from response objects M_{ij} to construct requirements matrix (tests $T = \{t_1, \ldots, t_n\}$ as stimuli; structural requirements $R_{\text{struct}} = \{r_1, \ldots, r_m\}$ as coverage elements; mutants $R_{\text{mut}} = \{m_1, \ldots, m_p\}$ as additional requirements; unified requirements $R = R_{\text{struct}} \cup R_{\text{mut}}$)
- For each test t_i, let $R_i \subseteq R$ denote the set of requirements covered by t_i (including mutants killed)
- Optional input: Reserved parameter $\alpha \in [0, 1]$ (default 0.3, currently unused; reserved for test execution cost weighting)
- Parameters: Mutation weight $\beta \geq 0$ (default 0.7); when $\beta = 0$, reproduces Enhanced HGS baseline [6]
- Output: Reduced SRM M' containing selected stimuli (rows) with original schema preserved

Algorithm

Procedure MAEHGS(M, β)
Input: Requirements matrix $M \in \{0, 1\}^{n \times m}$ where $M_{ij} = 1$ indicates test

t_i satisfies requirement r_j (unified structural and mutation requirements); mutation weight $\beta \geq 0$

Output: Minimal (heuristic) subset $S \subseteq T$ preserving all requirements

Phase 1: Essential Test Selection

1. Compute requirement cardinalities: $|T_j| = |\{t_i \mid M_{ij} = 1\}|$ for all $r_j \in R$
2. Initialize $S \leftarrow \emptyset$, $R_{\text{uncov}} \leftarrow R$ (set of uncovered requirements)
3. For each requirement r_j with $|T_j| = 1$:
Let t_i be the unique test covering r_j (i.e., $M_{ij} = 1$)
Add t_i to S (if not already added)
Remove all requirements covered by t_i from R_{uncov}: $R_{\text{uncov}} \leftarrow R_{\text{uncov}} \setminus R_i$

Phase 2: Weighted Greedy Selection

4. While $R_{\text{uncov}} \neq \emptyset$:
4.1 For each remaining test t_i not in S:
Let $\text{newCov}_{\text{struct}} = R_i \cap R_{\text{uncov}} \cap R_{\text{struct}}$ (uncovered structural requirements)
Let $\text{newCov}_{\text{mut}} = R_i \cap R_{\text{uncov}} \cap R_{\text{mut}}$ (uncovered mutant kills)
$\text{Score}(t_i) = |\text{newCov}_{\text{struct}}| + \beta \cdot |\text{newCov}_{\text{mut}}|$
4.2 Select $t_{i^*} = \arg\max_{t_i} \text{Score}(t_i)$
(ties broken by total new coverage $|\text{newCov}_{\text{struct}}| + |\text{newCov}_{\text{mut}}|$)
4.3 Add t_{i^*} to S
4.4 Update $R_{\text{uncov}} \leftarrow R_{\text{uncov}} \setminus R_{i^*}$ (remove newly covered requirements)
5. Return S

Complexity

Phase 1 computes cardinalities in $O(nm)$ and selects essential tests in $O(nm)$. Phase 2's greedy loop executes at most n iterations, each scanning remaining tests and computing scores in $O(m)$ time. Overall complexity is $\mathbf{O(nm)}$ for both time and space, matching the analysis in Chapter 5. When $\beta = 0$, the algorithm reproduces the empirically validated Enhanced HGS baseline [6].

A.1. Extended Benchmarks and Reproducibility

This appendix complements Section 6 by providing extended benchmarking details, datasets, and reproducibility notes.

Benchmark Tables

Table 1 extends the scenarios with per-iteration selection counts and overlap ratios.

Table 1.: Extended empirical benchmarks with overlap statistics

Scenario	Orig.	Min.	Red.	Cov.	Mean overlap
Large-scale test	20	9	55.0%	94.0%	0.36
Code coverage	6	4	33.3%	100.0%	0.18
Mutation testing	5	3	40.0%	100.0%	0.22
SRM conversion	5	3	40.0%	100.0%	0.20
Weighted (exec time)	6	3	50.0%	83.3%	0.41

Reproducibility Notes

Experiments are executed on an *Intel Core i7* with 16 GB RAM. Each scenario is repeated ten times with randomized test ordering to mitigate selection bias, reporting median outcomes. Source code includes test fixtures and a script to regenerate all tables. Random seeds and configuration parameters (e.g., α) are documented alongside results.

B.2. SRM Coverage Extraction Specification

We specify the process for extracting coverage information from LASSO Stimulus–Response Matrices (SRMs) for use by the minimisation algorithm:

B.2.1. SRM Structure

LASSO SRMs are two-dimensional matrices where:

- Rows (stimuli) represent test sequences, method invocations, or API calls

- Columns (systems) represent executable software units under test
- Cells contain structured response objects M_{ij} with execution results from system j responding to stimulus i

B.2.2. Coverage Extraction Process

For a selected target system (column j):

1. **System Selection:** Choose target system index j from SRM columns (current implementation: single system; multi-system minimisation is future work)
2. **Response Parsing:** For each stimulus i, extract per-test coverage metrics and mutation data from response object M_{ij}:
   - Per-test coverage across JaCoCo's 30 metrics (INSTRUCTION, LINE, BRANCH, METHOD, CLASS, COMPLEXITY)
   - Per-test mutation kills (individual mutants detected by this specific test)
   - Execution outcome (pass/fail status)
   - Optional: execution time for cost-aware minimisation
3. **Requirement Mapping:** Construct test–requirement relationships:
   - Tests $T = \{t_1, \ldots, t_n\}$ map to stimuli (rows)
   - Requirements $R = R_{\text{struct}} \cup R_{\text{mut}}$ where:
     - R_{struct}: structural coverage elements (statement/branch/method IDs)
     - R_{mut}: mutant IDs (prefixed with `"MUTANT:"`)
   - Coverage indicator: test t_i covers requirement r_k iff response object M_{ij} indicates per-test coverage of element k
4. **Validation:** Filter invalid or inconsistent stimuli (e.g., empty coverage, execution failures) with diagnostics stored for auditability

B.2.3. Output Format

The minimizer produces a reduced SRM M' where:

- Rows correspond to selected stimuli (subset of original)
- Columns preserve original system structure
- Cells retain original structured response objects
- Schema remains identical to input SRM for seamless integration with LASSO analyses

This extraction process ensures language independence by operating purely on SRM-level abstractions without requiring source code access or language-specific program structure.

C.3. Licensing Header Template

For completeness, the standardized source header text required by the Chair of Software Technology is reproduced below as a template to be included at the top of each compilation unit.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, December 1, 2025

Laith Philipp Sandouk

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, December 1, 2025

Laith Philipp Sandouk